

Proyecto 2

Generador de Analizadores Sintácticos

Descripción

Esta actividad culmina los contenidos del curso para crear un generador completo de analizadores léxicos (**YALex**) y sintácticos (**YAPar**). Se enfoca en implementar un ecosistema funcional que incluya la generación de tablas de parseo LALR, SLR(1) y LL(1), la capacidad de analizar sintácticamente un archivo de cadenas, y una interfaz gráfica tipo IDE para facilitar la interacción con el sistema.

- **YALex:** Generador de analizadores léxicos que produce tokens a partir de especificaciones regulares.
- **YAPar:** Generador de analizadores sintácticos que construye un analizador funcional basado en gramáticas y tokens proporcionados por YALex.

Objetivos

Generales:

- Desarrollar un generador de analizadores sintácticos funcional.
- Implementar la teoría de autómatas y gramáticas en un entorno práctico.
- Comparar las técnicas de análisis sintáctico LL(1), LALR y SLR(1) para demostrar las ventajas de cada enfoque.

Específicos:

- Implementar YALex para producir tokens en su totalidad desde un archivo de especificaciones regulares (esto ya lo hicieron en el proyecto anterior, pero lo necesitan para este proyecto también).
- Implementar la construcción de un Autómata LR(0) en su totalidad.
- Implementar analizadores sintácticos funcionales para ambas técnicas (LL(1), SLR(1) y LALR) en su totalidad .

Especificaciones

El programa debe cumplir con las siguientes especificaciones:

- Procesar un archivo con especificaciones regulares en YALex para producir tokens.
- Procesar un archivo con especificaciones gramaticales en YAPar para construir los analizadores sintácticos.
- Implementar **tanto LL(1) como SLR(1) y LALR**, siendo LALR o SLR(1) el método oficial a utilizar en futuras implementaciones del compilador.
- Evaluar un archivo con cadenas de entrada para determinar su corrección sintáctica usando ambos métodos.
- Proporcionar una interfaz gráfica tipo IDE con las siguientes funcionalidades:
 - Cargar archivos .yalex, .yapar, y archivos de cadenas de entrada
 - Editor de texto integrado para modificar especificaciones
 - Visualización de autómata generado LR(0)
 - Visualización de tablas de parseo (LL(1), SLR(1) y LALR)
 - Panel de resultados mostrando el análisis sintáctico
 - Guardado de archivos modificados
 - Manejo de errores con mensajes claros y ubicación en el código fuente

Entrada:

- Archivo .yalex con especificaciones regulares.
- Archivo .yapar con especificaciones gramaticales.
- Archivo con cadenas de entrada a analizar.

Salida:

- Visualización del automata generado LR(0)
- Tablas de análisis sintáctico (LL(1), SLR(1) y LALR)
- Resultados del análisis de las cadenas de entrada (aceptadas/rechazadas)
- Reporte de errores sintácticos y gramaticales encontrados

Funcionamiento del Programa

1. El usuario carga los archivos necesarios (.yalex, .yapar, y cadenas de entrada) a través de la interfaz gráfica.
2. **YALex** procesará las especificaciones regulares para generar un conjunto de tokens que serán utilizados en el análisis sintáctico.
3. **YAPar** tomará estos tokens y la gramática definida para:
4. El programa intentará analizar las cadenas de entrada usando **ambos métodos** (LL(1), SLR(1) y LALR)
5. El usuario podrá editar los archivos directamente en la interfaz y ejecutar el análisis.

Entregables

- Un repositorio de GitHub
- Un video no listado de YouTube (10 minutos) mostrando Demostración completa de la interfaz gráfica.
- Un informe PDF con todo el análisis y diseño de su proyecto. Con todo y los links correspondientes.

Evaluación

Requisitos para Calificación

Para poder optar a calificación, el programa debe funcionar correctamente el día de la presentación. El proyecto debe estar correctamente documentado y organizado.

Componente	Descripción	Puntos
Interfaz gráfica	Interfaz vistosa, intuitiva y fácil de utilizar con todas las funcionalidades requeridas (IDE completo)	10 pts
Manejo de errores	Correcto reporte y recuperación de errores sintácticos y gramaticales con ubicación precisa y mensajes descriptivos	10 pts
Comprensión del código	Preguntas sobre el código desarrollado. El código debe aceptar el análisis sintáctico de clases, funciones, recursión y listas. Mínimo 2 intervenciones por integrante y/o participación.	40 pts
Conocimiento teórico	Preguntas teóricas sobre el tema de análisis sintáctico. Mínimo 2 intervenciones por integrante.	40 pts
Extra	Corrección de problemas en vivo durante la evaluación y/o haber implementado funcionalidades adicionales más allá de lo solicitado en el proyecto	10 pts
Total		100 pts

Penalizaciones

- **Respuestas incorrectas:** Si un integrante responde incorrectamente a una pregunta, no recibirá los puntos correspondientes a esa pregunta. Si otro compañero del equipo responde correctamente la misma pregunta, ese compañero recibirá los puntos individuales.
- **Nota importante:** Las calificaciones de las preguntas (tanto de código como teóricas) se asignan de forma individual según las respuestas de cada integrante.